One assistant across your devices

Start with a personal exercise coach that sees, explains, demonstrates and remembers.

Your long-term vision is feasible as a family of connected applications and small device clients. Each device contributes what it does well: a phone sees, a watch supplies permitted activity data, a TV shows demonstrations, and a small board handles a microphone or sensor. They share an assistant service and controlled memory.

For Build North, demonstrate one complete experience: phone camera + one supported exercise + a reviewed 3D demonstration on a second screen + a preference remembered across sessions. The broader home assistant becomes the roadmap.

The first user and problem

Target an adult beginner who exercises at home and has difficulty turning written feedback into a movement they can understand. The proposed value is a short visual explanation at the right moment, with continuity across devices. Validate this with a few actual users before claiming demand.

A credible 90-second demonstration

• Sign in on the Expo phone app. Choose push-ups and a side-view camera setup. Set a preference such as short explanations.

• Pair a laptop display by scanning its QR code. A TV connected to that laptop can serve as the large display.

• Show a supported, visible form issue. The app gives one brief cue and plays the matching reviewed animation.

• Repeat the movement. The app checks again, or explains that the view is unclear. It does not automatically declare success.

• Reopen the session on the second device. Show the remembered preference and session summary, then edit or delete a memory.

Keep the claim honest

This document is a researched design proposal. No model, latency, exercise accuracy, cost or sponsor eligibility has been tested for your implementation. All acceptance targets below are proposed engineering gates, not achieved results.

Research method: 70 Exa search results, 69 unique result URLs, across model/vision, device support, sponsor infrastructure, comparable products, and validation research. Key official pages were read directly. Sources and their limits appear at the end.

What each device can realistically do

Device	Useful role	Implementation and limit
Phone / tablet	Live coaching, camera setup, personal memory, reminders.	Expo iOS/Android app. Use a development build for native streaming integrations. Start with one real phone.
Mac / Windows	Full assistant, session review, model settings, large visual display.	Ship the Expo web app in a browser first. Native desktop requires a wrapper or separate React Native desktop work.
TV	Exercise avatar; documentaries and shows based on preferences and available time.	First use a laptop connected by HDMI. Android TV / Apple TV can use React Native TV with Expo, but library support varies. Samsung/LG need separate assessment.
Watch	Read permitted sleep/activity records; timers, reminders and short controls.	Start by reading existing data through the phone. Dedicated watchOS/Wear OS interfaces are separate development work.
Fridge	User-confirmed food inventory, date reminders, meal ideas using ingredients.	Start with a phone scan or a tablet/Pi dashboard. An arbitrary fridge may not allow custom apps or sensor access.
Raspberry Pi	Home gateway, voice endpoint, display, local device integrations.	Linux service or browser dashboard. Treat it as a client/gateway; establish its actual inference performance before promising local video coaching.
ESP32 / ESP32-S3	Wake word, button, microphone, temperature/door sensor, small display.	Assuming this is what you meant by ESP40. Run firmware and small models, then call a nearby or cloud service. It cannot host the proposed 9B model.

Expo supports the mobile/web core; TV and desktop need platform-specific consideration. React Native lists Windows, macOS and TV as separate out-of-tree projects. [3-6]

Apple HealthKit and Android Health Connect require access to the requested data types. Imported sleep records are not a diagnosis. Food reminders should use dates the user confirms; a picture cannot establish that food is safe. [13-14]

MiniCPM-o is plausible; prove the whole loop

The official MiniCPM-o 4.5 model is a 9B audio/video/text model with full-duplex streaming: it can receive input while producing a response. Its model card lists Apache-2.0 licensing for code and weights. Verify the exact checkpoint and dependency licenses when you pin the build. [1-2]

Question	Practical answer
Can it run in the Expo app?	Use a server endpoint for the first version. A native mobile UI does not imply that the full model runs on the phone.
What hardware should I start with?	The official PyTorch demo asks for Linux and more than 28 GB VRAM. A 48 GB GPU is a reasonable initial candidate if you have access, but throughput and concurrency still need measurement.
Can it run locally on a desktop?	The project documents quantized desktop paths. Treat streaming, audio and tooling support as separate capabilities to verify for each runtime.
Does a fast model benchmark prove a fast app?	No. Camera capture, transport, queueing, inference, speech and rendering all add delay. Measure capture-to-cue timing on the intended phone and network.
Can any model replace it?	Only if it supports the required capability. A text model can plan or retrieve memories; it cannot automatically replace streaming visual perception.

Recommended roles

• MiniCPM-o: recognize the supported activity, understand the spoken question and explain an approved observation in plain language.

• MediaPipe Pose: estimate visible body landmarks and track their movement over time. Do not treat its estimated 3D coordinates as precise body measurements. [9-10]

• Application rules: decide whether evidence is sufficient, select a permitted cue, and control the matching animation.

Two implementation levels

First prove the official demo. Test a live session on the exact server before building a custom interface. Then build a bridge between your app transport and the model protocol. LiveKit can transport the phone media, but it does not itself implement MiniCPM inference. [1,8]

Fallback: if continuous streaming is unreliable, offer an explicitly labelled short-clip review mode. A sampled-image or upload workflow is useful, but it should not be described as validated full-duplex coaching.

A correction must be grounded in what is visible

Choose one exercise: side-view push-ups. Start with one reviewed alignment cue, correct examples, and an unclear-view state. Add another cue or exercise only after the first one passes the evaluation. The camera angle and occlusion materially affect pose estimates. [10]

Stage	Behaviour
1. Set up	Ask for a supported camera angle and enough space to see the relevant body parts. Show a framing guide before coaching starts.
2. Observe	Use timestamped frames, landmarks and short temporal windows. Track visibility and freshness. One awkward frame must not trigger a correction.
3. Decide	Apply calibrated rules for the selected exercise and view. If landmarks are missing, views are unsupported, or signals conflict, request a better view.
4. Explain	Choose one short cue. Describe the visible movement without claiming a diagnosis, exact joint loading or guaranteed injury prevention.
5. Demonstrate	Select a reviewed animation by an allowlisted ID. Speech, on-screen text and motion must describe the same cue.
6. Recheck	After the user tries again, reassess. Recognize an improvement only when the same observable condition supports it.
7. Remember	Save a brief, attributable session summary and a preference the user can change. Do not permanently label a person from one uncertain observation.

Output contract

Return a small structured action with an exercise ID, decision (observe / cue / unclear / pause), cue ID, animation ID, frame timestamp, expiry time and evidence references. Validate it before rendering or speaking. Drop stale actions after reconnects or exercise changes.

Full-duplex audio needs special care

MiniCPM can speak proactively. If speech plays before the app validates it, an output gate cannot correct what the user already heard. For the MVP, route coaching through approved cue text and controlled speech playback. Add freely generated streaming advice only after its behaviour is evaluated.

Avoid a promise of "any health tips". Keep the product focused on supported exercise guidance. Reports of pain or discomfort should pause the coaching instead of encouraging the user to continue.

Shared identity, selective memory, one session owner

Build one small backend with clear interfaces. Keep the model worker separate from the application API. Multiple devices can participate in the same session without each running an independent planner.

Component	Responsibility
Expo client	Login, permissions, session controls, camera publishing, memories and reminders.
Clerk + backend authorization	Verify the signed-in person. Derive the user ID on the server and check ownership of every session and paired device.
Session coordinator	Maintain current exercise, active devices, selected cue and one speaker. Issue expiring device/session tokens and broadcast state.
Perception worker	Receive media, run pose and model inference, return timestamped observations. Bound queues and discard stale frames.
SurrealDB Agent Memory	Store and retrieve durable preferences and attributable summaries for the correct user/context.
Structured application state	Store device capabilities, permissions, reminder times and completion status. Memory retrieval is not a reliable task scheduler.
Display client	Receive only the permitted session view. Play the chosen avatar animation; avoid exposing the user's full private history.

How Clerk fits naturally

Clerk handles sign-in and session identity. Your backend adds device pairing: the TV shows a short-lived code, the signed-in phone approves it, and the server issues a restricted display token. Clerk does not automatically implement your IoT permissions or data isolation. [11]

What shared memory means

Example: the user says "Keep feedback short." Save it with its source and time. The next session retrieves that preference, including on a different device. A screen shows why the assistant used it and lets the user correct or remove it.

SurrealDB distinguishes contexts, scope tags and permissions. A scope filter alone is not authorization. Use isolated demo contexts or properly enforced grants; test with two accounts. The public scope documentation and SDK examples are evolving, so pin the API/SDK and verify their actual selector shape. [12]

Default to no retained raw video. Check logging and session-recording defaults in the model demo before exposing it. Device-specific permission should determine which memories a shared screen can access.

Make the demonstration the main interface

The character should make an explanation easier to follow. Its motion is a product asset that you review, not something an LLM invents live.

A dependable 3D workflow

• Create or license one rigged character. Author or import a small set of exercise motions in Blender, then check them with a knowledgeable reviewer.

• Export the character and clips as GLB/glTF. Use stable animation IDs such as pushup_reference_side. Keep licenses and attribution with the assets.

• Use Three.js / React Three Fiber for the browser display. Expo-native rendering is documented, but introduces platform and performance work. Prove the large-screen version first. [15]

• Keep lighting simple, reuse one model, reduce texture sizes, and pause rendering when the display is inactive. Measure on the actual target device.

• If native 3D is unstable, show a pre-rendered video of the same reviewed character on the phone while the browser display retains interactive 3D. Make that distinction clear.

Four screens are enough

Screen	Essential controls
Today	Start session, resume a plan, next reminder. One clear primary action.
Coach	Framing preview, concise cue, avatar, pause/mute controls, and visible camera status.
Memory	What was remembered, source/date, edit/delete, and what can appear on shared devices.
Devices	Pair or revoke a screen; show whether it can view, listen, speak, or receive private context.

Make the assistant calm

Give one correction at a time. Avoid repeating it on every frame. Provide a "show me again" control and let the user choose text, audio or demonstration. When evidence is weak, show "I need a clearer view" rather than a fake confidence percentage.

The phone remains the primary control device while the larger screen shows the movement. Only one device speaks at a time, which avoids echo and conflicting instructions. Treat microphone echo cancellation and interruption handling as features to test.

Give each sponsor a real responsibility

Build North asks for a specific user, practical value and credible deployment. Sponsor awards require meaningful platform use; Best Overall does not require every sponsor. [21]

Platform	Strong use in this project	Evidence to show
Expo	Phone app with login, camera session controls, memories and pairing.	A working build on a real device and a documented launch path.
Clerk	User identity across sessions; backend maps devices and permissions to the authenticated user.	Two accounts remain separated; sign-out or device revocation prevents future access.
SurrealDB Agent Memory	Recall user-approved preferences and past session summaries across devices.	Change a preference, reopen elsewhere, show its source and edit/delete behaviour.
Era by Eon	Optional tool-using workflow for a fitness coach or small studio, tested against its simulated business systems.	Repeatable tasks with answer keys, logs, model/prompt comparisons and a separate evaluation environment.

Era is not a personal video fine-tuning service

Era supplies synthetic companies, connected records and compatible APIs/MCP tools. Its documentation describes testing and benchmarking, and using graded episodes in your own post-training pipeline. This does not establish built-in training for exercise video or personal health models. [16]

An optional branch that genuinely fits Era

A fitness coach asks the assistant to review a fictional client's support requests, reconcile them with their membership record, and draft a session-preparation summary. Use the systems Era actually exposes, such as CRM, support and shared documents. Test whether the assistant cites the right records, handles contradictions and requests approval before an external action.

This branch can support a later studio product. It should be included this week only if it strengthens the same user story and the core coaching loop is already stable. Custom household or fitness scenarios must be confirmed with Eon; do not assume their availability.

Personalization before training

Start with explicit preferences, scoped memory, retrieval and reviewed demonstrations. Fine-tuning is a later experiment for a measured recurring failure. Use consented/licensed data and a held-out test set; a person's changing preferences should normally remain editable data.

Measure whether the assistant helps

Freeze a modest evaluation before optimizing. Suggested starting point: 20 development clips plus 60 separate test clips: 20 acceptable executions, 20 supported issues, and 20 unclear views or out-of-scope cases. These are proposed counts; use fewer if needed and state the actual sample size.

Keep people and recording sessions separate between development and test wherever practical. Adjacent frames from the same video are not independent examples. Label clips with a knowledgeable reviewer and record disagreements; do not call unreviewed labels expert ground truth.

Measure	How to report it
Correction precision	Correct supported cues / all cues issued. Also report false cues on acceptable executions.
Supported-issue recall	Detected target issues / labelled target issues. Pair this with precision to expose an overly silent system.
Unclear-view handling	Correct abstentions on unclear clips, plus unnecessary abstentions on clear clips.
End-to-end delay	Frame capture to first valid cue: median and p95, measured on the real phone, server and network.
Demonstration agreement	Whether cue text, spoken audio and animation convey the same approved correction.
Cross-device continuity	Preference recall, edit/delete behaviour, session handoff and correct screen permissions.
Operating cost	GPU runtime, provider usage and transport cost per completed session, including failed attempts.

Suggested initial gates - not results

Aim for at least 90% correction precision on the narrow held-out set, at least 90% correct abstention on the unclear subset, and p95 capture-to-cue below 3 seconds on the demo setup. Report raw counts alongside percentages. These gates guide scope decisions; passing them is not medical validation.

Require zero cross-account memory disclosures in the explicit isolation tests, correct device revocation, and no duplicate reminders after reconnect/retry. These are functional requirements, not statistical guarantees of security.

Compare the right alternatives

Compare pose/rules alone, MiniCPM alone, and the combined pipeline on identical clips. Separately compare memory on/off for preference continuity. Use Era only for supported tool workflows; use the labelled video set for coaching. Save prompts, model revisions, settings and failure examples with each run.

Ship the smallest complete experience first

Dates below are Toronto time. The official Devpost page states a final submission deadline of Sunday, October 25, 2026 at 11:59 PM ET; its countdown labels the midnight boundary as October 26 at 12:00 AM. [21]

Day	Deliverable	Decision gate
Mon Oct 19	Run official MiniCPM demo on available hardware; test the actual phone/network. Confirm access and asset licenses.	If live streaming cannot be demonstrated, choose explicit clip-review mode early.
Tue Oct 20	Expo app, Clerk sign-in, backend, memory write/read, two-account isolation test.	A preference survives reopening and is inaccessible to the second account.
Wed Oct 21	One camera path, server receiver, pose extraction, one supported exercise and unclear-view handling.	A real phone produces timestamped observations; no stale frame backlog.
Thu Oct 22	Reviewed avatar motion, matching cue, phone-to-display pairing and recheck loop.	Complete the end-to-end demo without manually selecting the answer.
Fri Oct 23	Freeze and run evaluation; repair the largest failure classes.	Publish actual counts and latency. Reduce scope if a supported cue is unreliable.
Sat Oct 24	Deployment, test access, error states, demo recording and write-up. Era only if already well integrated.	Another person can open or install the demo from the instructions.
Sun Oct 25	Final submission check. Target your own submission by 8 PM for a buffer.	Public repo, working link, team names, disclosures and measured limits are present.

If time runs short, cut in this order

Remove native watch/fridge/desktop apps, the second exercise, hardware, per-user fine-tuning and the optional business workflow. Keep one phone, one display, identity, memory and one demonstrable correction. If only the display works, describe it as a web prototype and do not claim a completed mobile integration.

The Devpost demo video is optional and limited to four minutes. A focused video under two minutes is a useful choice for this story. Top-ten notification is October 26 and the in-person Toronto demo is October 29; at least one teammate must attend. [21]

Make continuity the reason to choose it

Camera-based exercise guidance already exists. Tempo describes form feedback, Peloton IQ includes camera-assisted guidance, and Apple Workout Buddy uses workout history and current activity for spoken encouragement. Those products show overlap; they do not establish that your particular implementation will work. [17-19]

Your proposed differentiation

Combine visual teaching, user-visible memory, device handoff and model choice without requiring a proprietary fitness machine. Treat the benefit as a hypothesis: can a user understand and apply a cue more easily, and do they want to return to the next session?

Stage	Product branch	What unlocks it
Now	At-home movement coach for one supported exercise.	A useful live loop, measurable feedback, and repeat use by a few testers.
Next	Daily routine assistant: user-chosen exercise reminders, sleep summaries and TV suggestions.	Permissioned integrations, clear separation between observation and user preference, and reliable scheduling.
Later	Kitchen companion and household devices.	User-confirmed food inventory; Pi or ESP32 endpoints; capabilities, pairing, revocation and household profiles.
Business option	Companion for independent fitness coaches or small studios.	A coach dashboard, client-approved sharing, verified workflows and a pilot customer. Era can test the tool workflow.

One home example, with evidence boundaries

The watch supplies an authorized sleep summary. The phone asks whether the user prefers a shorter session. The TV displays the chosen routine. A kitchen reminder refers to a date the user entered. Each action explains its basis; uncertain sensor readings do not become permanent personal facts.

Cost model and deployment

Estimate monthly cost as GPU-hours x actual rate + hosting + transport + memory/API usage + storage. Do not assume an idle GPU is free, that sponsor hosting credits include GPUs, or that one streaming worker supports many people. Measure a ten-minute session and then estimate usage.

Use one GPU worker, one lightweight API service, one memory deployment and a web display initially. Bound session length and idle time. For a local-only option later, check every dependency: local model inference with hosted authentication or hosted memory is still a hybrid system.

Instructions for the coding agent

Treat the following as the project brief. Ask the agent to inspect the existing repository before choosing package versions or replacing any working components. Do not assume prior work is eligible to be presented as new.

• 1. Prove the risky dependencies. Run the official MiniCPM streaming demo on the available server, establish a real phone connection, and record setup and timing. Pin the model and dependencies.

• 2. Build the smallest vertical slice. Expo client, verified Clerk identity, a backend session and one SurrealDB Agent Memory preference. Test with two accounts before attaching personal data.

• 3. Define typed boundaries. DeviceCapabilities, SessionState, Observation, CoachAction, MemoryEntry and Reminder. Keep model-specific protocols behind an adapter.

• 4. Implement one media path. Use an Expo development build with a supported WebRTC integration if native streaming is required. Build the server-to-MiniCPM bridge explicitly. Do not assume expo-camera is the bridge.

• 5. Implement one exercise. A supported side view, temporal evidence, one reviewed cue and abstention when the relevant landmarks are not visible. Keep thresholds configurable and calibrated from development data.

• 6. Gate outputs. Validate actions against a cue/animation allowlist and the current exercise/session. Control speech playback through the same action. Reject stale, duplicated or conflicting outputs.

• 7. Pair a display. Use a short-lived code approved on the authenticated phone. Issue a restricted display token. Support revocation and one speaking device.

• 8. Make memory editable. Store source, timestamp, scope and whether a statement is user-confirmed or inferred. Test changes and deletion. Keep authoritative reminders and permissions in structured state.

• 9. Evaluate and document. Use a frozen held-out clip set and an explicit two-account device test. Report real counts, latency and failures. Label recorded demonstrations and mock data.

• 10. Deploy and submit. Provide a testable build or link, setup instructions, model/asset licenses, sponsor contributions and disclosures of earlier work and AI tools.

Definition of done

A new tester can sign in, start a supported session, receive a grounded cue and matching visual demonstration, pair a second display, and revisit an editable preference. The app handles denied camera permission, loss of connection and an unclear body view without inventing a successful result.

Technical sources and their limits

[1] OpenBMB MiniCPM-o demo
Official streaming demo, deployment requirements and worker architecture. Vendor timing is not an app benchmark.

[2] MiniCPM-o 4.5 model card
Model modalities, size, runtime paths and license. Performance claims are from the model authors.

[3] Expo documentation
Core mobile/web development path.

[4] Expo: TV builds
Apple TV/Android TV route and SDK limitations.

[5] Expo: additional platform support
Module-level macOS/tvOS support needs native adaptation.

[6] React Native: out-of-tree platforms
Windows/macOS/TV are separately maintained platform targets.

[7] Expo Camera
Camera preview, photo and video APIs; not proof of an end-to-end streaming integration.

[8] LiveKit: Expo quickstart
Expo WebRTC integration requires native code and a development build.

[9] MediaPipe: Pose Landmarker
Landmark and tracking capabilities. Does not establish coaching validity.

[10] Dill et al., 2023: pose accuracy
Original small study reporting exercise/viewpoint dependence. Not a broad validation of this proposed app.

[11] Clerk: Expo quickstart
Identity integration and token handling. App-specific authorization remains your responsibility.

[12] SurrealDB: contexts and scope
Distinction between context isolation, scope filtering and access grants. Verify the prerelease API against your deployment.

All links reviewed on 19 October 2026. Product documentation can change; pin versions when implementation begins. The recommendations and build schedule are design judgments, not claims endorsed by these sources.

Device, product and event references

[13] Apple: HealthKit authorization and sleep
Permission model. Sleep type: developer.apple.com/documentation/HealthKit/HKCategoryTypeIdentifier/sleepAnalysis.

[14] Android: Health Connect sleep experiences
Sleep records and permissions. Reading existing records differs from collecting them.

[15] React Three Fiber: installation
Browser and native rendering paths. Test stable compatible versions on physical devices.

[16] Eon: Era introduction
Synthetic enterprise environments for agent evaluation; graded episodes may feed a separate training pipeline.

[17] Tempo: 3D vision and form feedback
Official product capability description; not independent performance evidence.

[18] Peloton IQ
Official description of personalized training and form feedback.

[19] Apple Workout Buddy
Voice encouragement informed by activity history; platform requirements apply.

[20] ESPHome voice assistant
Example of small devices streaming audio to a backend. Home Assistant offers an established device-extension pattern.

[21] Build North Devpost
Submission deadline, judging, sponsor categories and deliverables.

[22] Build North rules and resources
Prior work/AI-tool disclosure and public submission requirements. Resources: build-north.devpost.com/resources.

[23] Espressif ESP32-S3-BOX-3
Microcontroller RAM and local speech-command capability; supports the endpoint/server distinction.

[24] Home Assistant voice architecture
Capture, interpretation and action can live in different components.

[25] SurrealDB TypeScript SDK
Agent Memory client, distinct from merely storing records in a general-purpose database.

The sponsor-linked Era hackathon PDF could not be read through the public fetch; conclusions about Era use its official product documentation and repository. Custom exercise-video evaluation or hosted fine-tuning was not verified.